

API как продукт: проектирование и управление

AI-Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Иван Четвериков

AI Architect

- Более 5 лет опыта в разработке и архитектуре
- Специалист в разработке и проектировании AI-приложений в Raft

 @istrebitel_1

 istrebitel.3.12@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический фундамент и планирование проекта

Проектирование и документирование

Инфраструктура

Продвинутые архитектурные паттерны

Качество, интеграции безопасность

Стратегия, лидерство и экономика

Проектная работа

ФИНИШ



Маршрут вебинара

API как продукт

Жизненный цикл API

Специфика AI API

Итоги

Q&A

Цели вебинара

К концу занятия вы сможете

1. Применять продуктовый подход к разработке API

2. Проектировать, документировать и управлять жизненным циклом API

3. Разрабатывать стратегию вывода внутреннего AI-сервиса в публичный API



Смысл

Это нужно уметь чтобы:

1. Рассматривать API как продукт с пользователями, ценностью и метриками
2. Создавать, документировать и сопровождать API на всех этапах жизненного цикла
3. Знать, как превратить внутренний API во внешний продукт



**Почему важно смотреть на
API как на продукт, а не как
на техническое решение?**



API как продукт

API как продукт - это

подход, при котором программный интерфейс рассматривается не просто как технический инструмент, а как самостоятельный коммерческий продукт



Кто потребляет ваш API?



Внутренние команды

Микросервисы, фронтенд, мобильное приложение. Главное — скорость и стабильность



Партнёры

B2B-интеграции. Нужны контракты, SLA, версионирование и поддержка



Внешние разработчики

Third-party, marketplace. Критичны onboarding и документация



Конечные пользователи

Через SDK, виджеты. Важны простота и минимальный time-to-value



Один API может обслуживать несколько типов потребителей одновременно

Принципы хорошего API-дизайна

Consumer-First

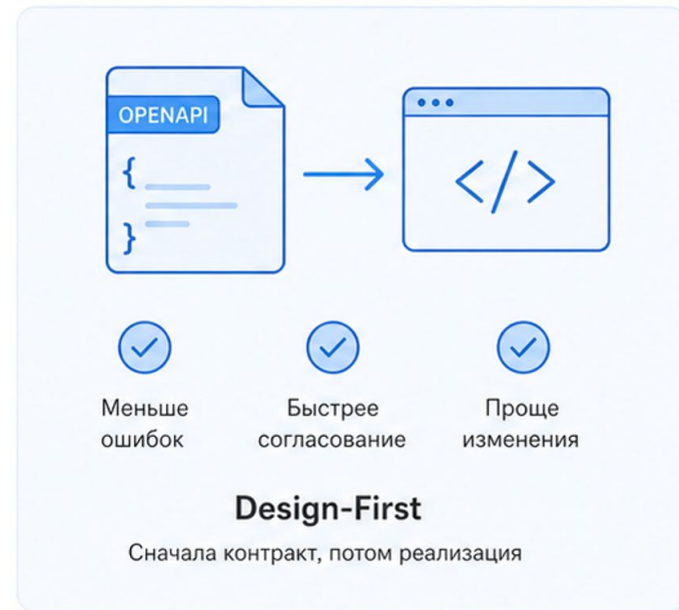
- Начинайте с вопроса "кто и как будет вызывать этот метод", а не "какие данные у меня есть в базе".



Принципы хорошего API-дизайна

Design-First

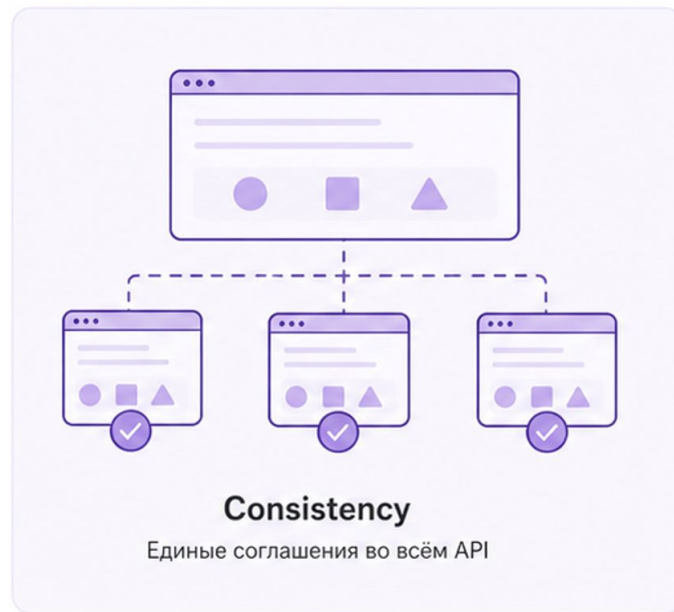
- Сначала пишем контракт в формате OpenAPI-спецификации, потом реализуем. Это снижает стоимость изменений в разы. Если вы обнаружили, что структура неудобна на этапе спецификации — исправить за 5 минут. Если на этапе готового кода — это дни работы.



Принципы хорошего API-дизайна

Consistency

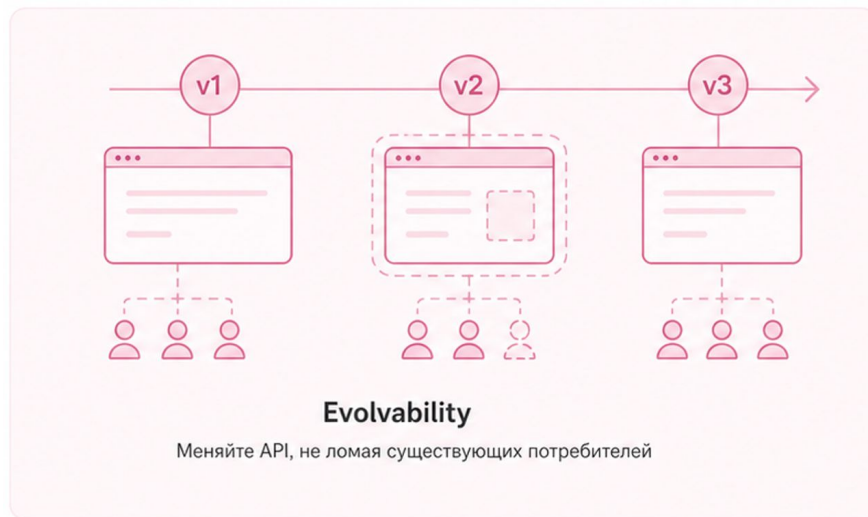
- Единые соглашения по именованию, структуре ошибок, форматам дат — во всех эндпоинтах. Потребитель не должен каждый раз "учить" новый стиль.



Принципы хорошего API-дизайна

Evolvability

- API будет меняться. Думайте об этом с первого дня: как вы будете вносить изменения, не ломая существующих потребителей?



Принципы хорошего API-дизайна

Discoverability

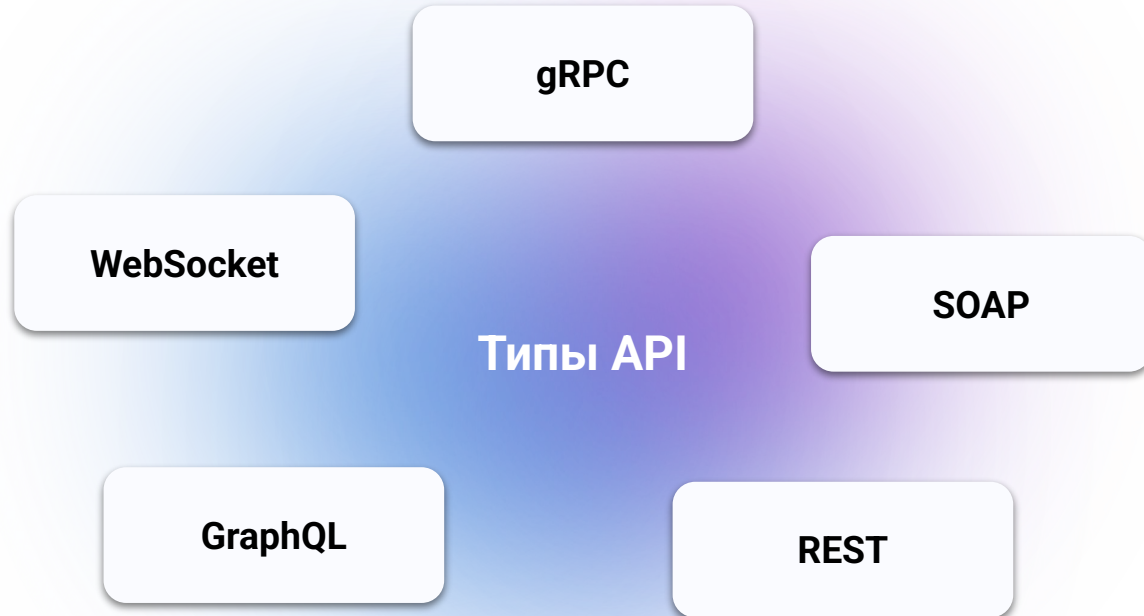
- Потребитель должен понимать, что умеет ваш API, без звонка в поддержку. Понятные названия методов, примеры, описания ошибок — все это часть продукта.



**Какие типы API вы знаете?
(REST, ...)**



Основные типы API-интерфейсов



REST API

Representational State Transfer

REST API построен на стандартных протоколах HTTP и обычно возвращает данные в формате JSON, но может поддерживать и разные форматы.

REST API использует стандартные HTTP-методы для взаимодействия (GET, POST, PUT, DELETE и т.д.). Каждый метод соответствует определенному типу действий, которые могут быть выполнены над ресурсами API.

Простота и гибкость REST API сделали его лучшим выбором для многих разработчиков, но при работе с большими объемами данных он может быть неэффективным, так как для получения всех необходимых данных может потребоваться несколько запросов.



SOAP API

Simple Object Access Protocol

SOAP - протокол для обмена структурированной информацией в веб-сервисах с использованием XML. Он обладает высокой расширяемостью и позволяет осуществлять обмен данными через различные транспортных протоколов, (HTTP, SMTP и другие).

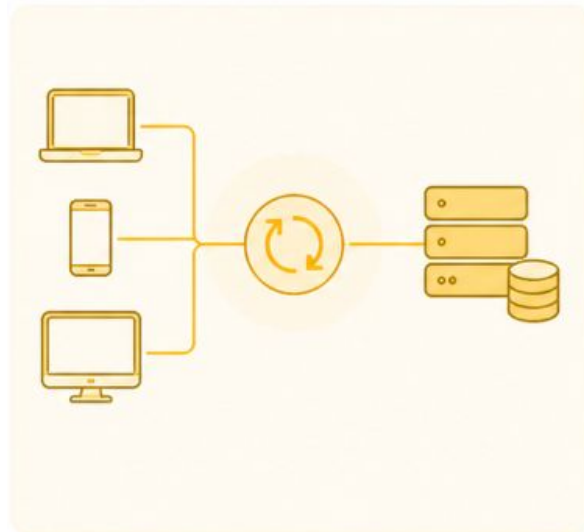
API SOAP известны своей надежностью и часто используются в энтерпрайз-разработке. Они предлагают встроенную обработку ошибок и их можно использовать с различными сетевыми протоколами. Однако зависимость SOAP от XML может приводить к большим объемам передаваемых данных, что делает его менее эффективным по сравнению с другими API в каких-то сценариях использования.



WebSocket API

WebSocket API постоянный канал связи между клиентом и сервером и сохраняет соединение открытым, что позволяет передавать данные в режиме реального времени.

Несмотря на преимущества, WebSocket API могут быть более сложными в реализации и требуют больше ресурсов для поддержания открытых соединений.



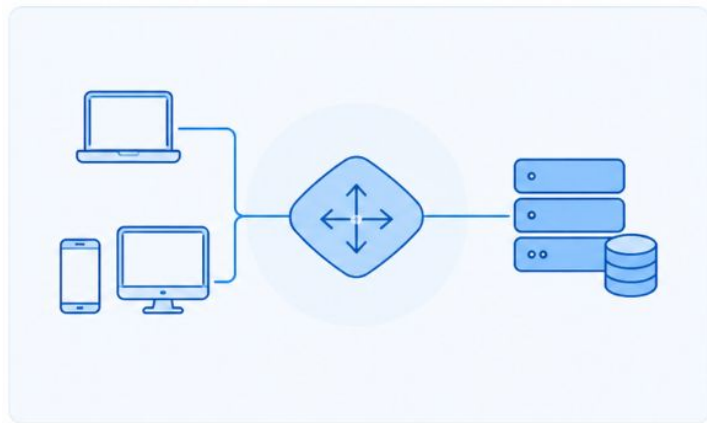
gRPC API

Google Remote Procedure Call

Высокопроизводительная платформа с открытым исходным кодом, разработанная компанией Google. Используется HTTP/2 для передачи данных и Protocol Buffers (protobuf), высокопроизводительный формат двоичных данных, в качестве языка определения интерфейса.

gRPC поддерживает четыре типа взаимодействия:

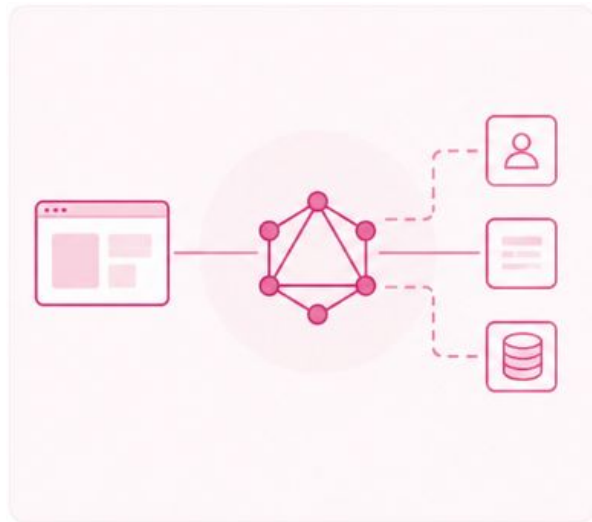
- унарный (запрос-ответ)
- потоковая передача данных с сервера,
- потоковая передача данных от клиента
- двунаправленная потоковая передача.



GraphQL API

Язык запросов к API и рабочую среду для выполнения этих запросов. В отличие от REST API, где вам нужно делать запросы к различным эндпоинтам, чтобы получить соответствующие данные, GraphQL позволяет сделать один единственный запрос, чтобы получить именно те данные, которые вам нужны.

В GraphQL пользователи определяют форму и размер ответа, что приводит к более эффективной загрузке данных и уменьшению избыточной выборки данных. Он отлично подходит для сложных систем и микросервисов, где данные распределены между различными сервисами.



Когда какой протокол можно выбирать

REST

Публичный API для внешних разработчиков.
При стриминге ответов (LLM, live-данные)
добавляем SSE.

WebSocket

Real-time двусторонняя связь.
Чаты, игры, оповещения.

gRPC

Микросервисы где важна скорость (в основном, внутренние для продукта).

GraphQL

Сложный UI-клиент с множеством вложенных запросов. Хорошо встраивается в паттерн BFF.

* SOAP → Legacy / enterprise интеграции



Вопросы



если есть вопросы



если вопросов нет

Жизненный цикл API

Стратегии версионирования API

- 1. URI Versioning:** /api/v1/predict, /api/v2/predict
Самый популярный для публичных API. Просто, понятно, легко кэшировать.
Рекомендовано для внешних API.
- Header Versioning:** Accept: application/vnd.api+json;version=2
2. Чистые URL, но сложнее в тестировании и документировании.
Используется в GitHub API.
- Deprecation Policy:** поддержка старой версии N месяцев после выхода новой
3. Предупреждайте через Sunset-header, Changelog, email-рассылку и т.п.
Минимум несколько месяцев до отключения старой версии.



Инструменты документирования API



OpenAPI / Swagger

Стандарт описания REST API. Автогенерация документации и SDK. Swagger UI — интерактивная документация.



Postman / Insomnia

Среда тестирования и коллекции запросов. Postman позволяет публиковать документацию и создавать mock-серверы.



Developer Portal

Центральный хаб: API-reference, guides, changelog, примеры кода, community. Примеры: Stripe Docs, Sphinx.

API Gateway - зачем нужен и что делает



Аутентификация и авторизация

API-ключи, OAuth 2.0, JWT. Разграничение доступа по тарифам



Rate Limiting

Защита от перегрузки. Квоты: запросов в минуту / в месяц по тарифному плану



Мониторинг и логирование

Метрики latency, error rate, usage. Основа для аналитики и биллинга



Версионирование и роутинг

Одновременная поддержка v1/v2. Плавный переход без downtime

Модели монетизации API



Freemium

Бесплатный tier с ограничениями (rate limit, объём данных).
Платные тарифы снимают ограничения.



Pay-per-use

Оплата за фактическое использование: запросы, токены, GB. Нет фиксированных затрат для клиента.



Subscription Tiers

Фиксированная оплата за уровень доступа (Starter / Pro / Enterprise).
Предсказуемая выручка.

Вопросы



если есть вопросы



если вопросов нет

Специфика AI API

Чем AI API отличается от обычного?

Уникальные вызовы при проектировании AI-сервисов для внешних потребителей

- **Недетерминированность:** одинаковый запрос может дать разный ответ. Нужны параметры (temperature, seed) и их объяснение в документации.
- **Высокая latency:** inference медленнее обычного API. Нужен streaming (SSE/WebSocket) и timeout-политика для улучшения UX.
- **Дорогой инференс:** стоимость зависит от модели и объема токенов. Rate limiting и кэширование критичны.
- **Версионирование моделей:** обновление модели = смена поведения. Нужна политика стабильности.

AI API = обычный API +
специфика ML:
недетерминированность,
высокая стоимость,
версии моделей



Мульти- тенантность в AI API

- 1. Изоляция данных по тенанту**
Каждый клиент (компания/пользователь) должен видеть только свои данные: историю запросов, файн-тюнинг, настройки. Реализация: `tenant_id` в JWT-токене или API-ключе + row-level security в БД.
- 2. Rate limiting на уровне тенанта**
Квоты разные для каждого тарифа: Starter — 100 req/min, Pro — 1000 req/min. Важно предусмотреть отдельный лимит на параллельные запросы, особенно для LLM-инференса.
- 3. Изоляция вычислительных ресурсов**
Enterprise-тенанты часто требуют dedicated inference (выделенный GPU/нода). Shared-инференс дешевле, но создаёт "шумных соседей". Решение – приоритетные очереди + burst limits.



Монетизация AI API



Оплата за токены

Input + output токены считаются отдельно. Output дороже input (x3–x5). Важно показывать расход в real-time и возможность посчитать стоимость до отправки запроса.



Предоплата “кредитов”, оплата за пакет

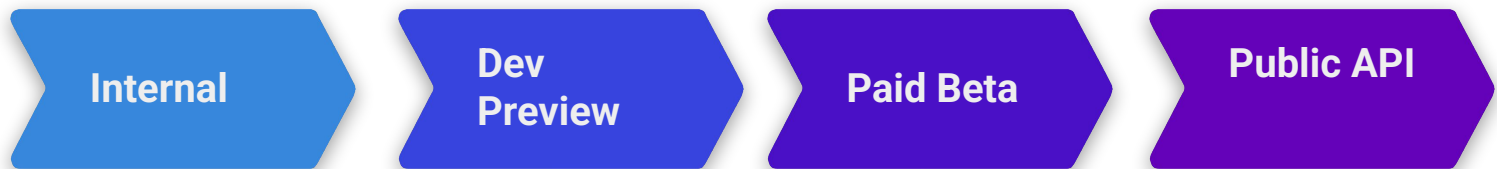
Клиент покупает пакет токенов/запросов заранее со скидкой. Снижает churn, улучшает cash flow. Burn rate дашборд обязателен.



Enterprise - кастомные правила

Скидка по объему + SLA + dedicated instances. Договорная цена. Обязательно верхний порог лимитов и overages policy, чтобы клиент не получил неожиданный счет.

От внутреннего AI-сервиса к публичному API



Только внутренние команды. Нет SLA, нет документации. Быстрая итерация, прямая связь с пользователями

Закрытый доступ для партнеров. Первая документация, базовый SDK. Сбор обратной связи

API Gateway, rate limiting, биллинг. Онбординг через Developer Portal. Первые реальные клиенты

Открытая регистрация. Полная документация, SLA, support SLA. Маркетинг и экосистема

OpenAI API vs Anthropic Claude API

OpenAI API

Запуск: 2020. Модель: pay-per-token.
Freemium для старта (убрали в 2024),
потом billing по API-ключам.

Сильный Developer Portal, Playground.
Rate limits по уровням. Экосистема plugins,
fine-tuning, DALL-E, Whisper. 100M+
запросов/день.

Claude API

Запуск: 2023. Упор на safety и long-context
(100k+ токенов).
Pay-per-token, Tier-based rate limits.

Фокус на enterprise: безопасность,
контроль. Developer Console для
тестирования.



Метрики успеха API-продукта



Adoption Rate

Сколько команд/клиентов реально используют API. Рост month-over-month



Time to First Call

Время от регистрации до первого успешного запроса. Цель: < 30 минут



API Reliability

Uptime + p99 latency. SLA для внешних потребителей: 99.9%+



Revenue / Usage

Для монетизированных API: выручка, активные платящие клиенты

Вопросы



если есть вопросы



если вопросов нет

Разбор кейсов проектирования

Потребители и ценность

AI База Знаний, Enterprise B2B

Клиент – “большой бизнес”.
Интегрируют API в юридическую систему:
высокий compliance.

Ключевые требования: SLA 99.9%+,
dedicated infrastructure, audit log запросов.
Ценностное предложение: “Сократите ревью
договора с 2 часов до 4 минут”. Данные не
должны обрабатываться вне страны.

Чат-бот тех.поддержки, SMB API

Клиент – малый/средний бизнес,
автоматизирующий время ответа
техподдержки своим клиентам.
Хочет подключить к Zendesk по
инструкции из N шагов за минимальный
ttm.

Ключевые требования: простота
онбординга, низкая стоимость. Уйдёт к
конкуренту, если что-то сложно.



Дизайн API. Эндпоинты и протокол

AI База Знаний → REST + async jobs

POST /v1/documents/analyze
body: { file_url, analysis_depth } → returns: { sections, risks, job_id }

GET /v1/jobs/{job_id}/status → polling до завершения (30–90 сек, SLA)

Паттерн: async job queue. Клиент хочет структурированный JSON — стриминг не нужен. Webhook при готовности.

Чат-бот тех.поддержки → REST + SSE streaming

POST /v1/reply/suggest
body: { ticket_text, tone } → returns: [{ text, confidence }]

POST /v1/reply/stream → SSE: токены по мере генерации. Параметр stream=true.

Клиент видит ответ "в реальном времени", что критично для UX.

Версионирование и управление изменениями

AI База Знаний → жёсткая политика

Deprecation: 12 месяцев.

Contract-first: RFC-документ перед каждым breaking change + отдельная staging-среда для валидации.

Причина: корпоративный клиент встроил API в систему, которую строили 2 года.

Breaking change = инцидент с финансовыми последствиями.

Чат-бот тех.поддержки → гибкая политика

Новые поля в ответе) не считаются breaking — клиентский код их игнорирует.

Deprecation: 30 дней. Email + заголовок

Поддержка старой версии: ~3 месяца.
SMB обновят интеграцию быстрее, чем Enterprise-клиент согласует внутренний RFC.



Документация и онбординг

AI База Знаний → Enterprise документация

OpenAPI-спецификация со всеми edge cases и обработкой ошибок.

Security guide: что логируется, где хранятся данные, какие scopes за что отвечают.

Compliance-документы: обработка ПДн и т.п.

Онбординг: 2–4 недели. Включает звонок с Solutions Engineer и sandbox-среду с тестовыми сценариями.

Чат-бот тех.поддержки → SMB онбординг

Quickstart: 5 шагов — "скопируй этот curl, получи первый ответ за 2 минуты".

Готовые плагины: Zendesk, Freshdesk, кнопка Install, без кода.

Видео-тutorиал 3 минуты. Ни один SMB-клиент не будет читать OpenAPI прежде чем попробует.

Time to First Call < 10 минут.



API Gateway: auth, rate limits, тенантность

AI База Знаний → Enterprise Gateway

Auth: OAuth 2.0 client credentials. client_id + secret → short-lived token (TTL 1 час) с tenant_id и scopes.

Rate limiting: по tenant_id, по контракту (напр. 500 документов/сутки, max 10 concurrent jobs). HTTP 429 + уведомление аккаунт-менеджеру.

Audit log: GET /v1/audit/logs — клиент видит все свои запросы.

Инфраструктура: dedicated inference-нода для Enterprise-типа. Изоляция от "шумных соседей", гарантия latency SLA.

Чат-бот тех.поддержки → SMB Gateway

Auth: API-ключи (X-API-Key: sk-...).

Rate limiting: по API-ключу и тарифу. Free — 100 req/сутки, Pro — 50 000/мес. HTTP 429 + ссылка на upgrade в теле ответа — конверсионный механизм.

Pool изоляция для всех.

Burst limit: 10 req/sec на ключ.

Тенантность минимальная: ключ видит только свою историю запросов.



Монетизация

AI База Знаний → ACV + overage

Annual Contract Value: N₽/год — включает обработку 3 000 документов/месяц + 1 dedicated instance.

Overage: M₽ за документ сверх лимита.

Почему не pay-per-use: корпоративный CFO хочет предсказуемый бюджет.
Непредсказуемые расходы не бюджетизируются.

Чат-бот тех.поддержки → Freemium + tiers

Free: 500 ответов/мес.

Starter N₽ : 5 000.

Pro M₽: 25 000 + стриминг + приоритет.

Ключевая метрика: конверсия Free → Paid.

Триггер апгрейда — hit rate limit в теле 429-ответа. Годовые контракты SMB не подписывают.



Сравнительная таблица решений

	Enterprise B2B База Знаний	SMB API Техподдержки
Потребитель	Enterprise клиент	SMB клиент
Протокол	REST + async job queue	REST + SSE стриминг
Auth	OAuth 2.0, short-lived tokens	API-ключи
Rate limiting	По контракту, tenant-уровень	По тиру
Тенантность	Dedicated instances, audit log	Pool, минимальная
Версионирование	12 мес. deprecation, RFC-процесс	~3 мес. deprecation, преимущество новым фичам
Документация	OpenAPI + compliance docs	Quickstart + видео + интеграции
Онбординг	2–4 недели, Solutions Engineer	< 10 минут, self-serve
Монетизация	Годовая лицензия + потребление	Freemium + тариф на месяц
Ключевой KPI	Продление контракта, соблюдение SLA	Free→Paid конверсия, нет оттока клиентов



Чему нас учат эти кейсы

1. Потребитель определяет все: протокол, документацию, монетизацию, уровень изоляции. Не проектируйте API до ответа на вопрос "кто мой потребитель и что для него провал?"
2. Enterprise и SMB — принципиально разные продукты. Один API для обоих сегментов будет неудобен для всех
3. AI-специфика влияет по-разному: Enterprise критична стабильность модели и async-обработка; SMB — стриминг и низкая стоимость за запрос



Вопросы



если есть вопросы



если вопросов нет

**Подведём
итоги занятия**

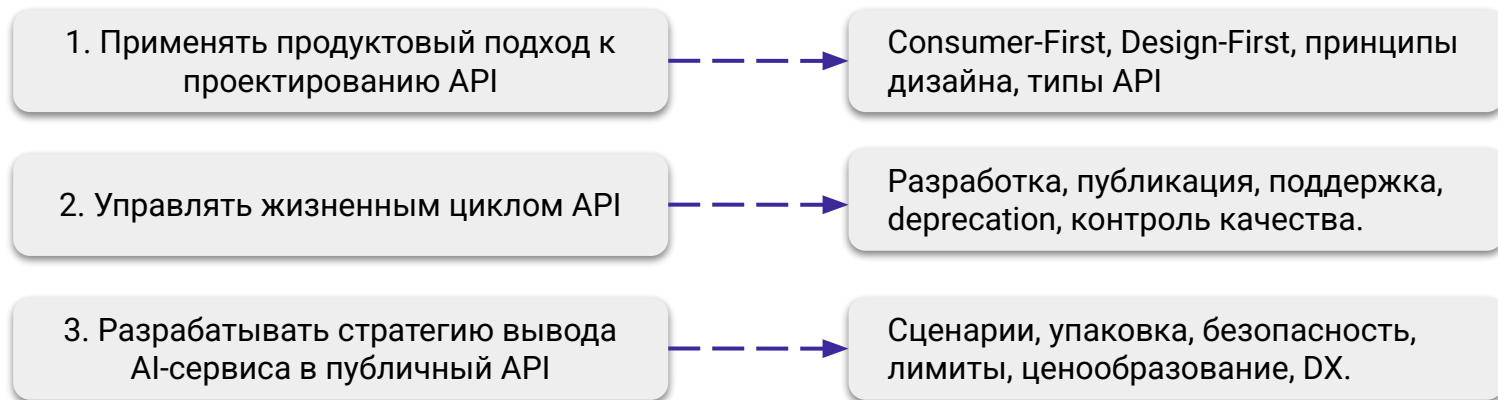
Ключевые тезисы занятия

1. API - это продукт. Проектируйте с точки зрения потребителя, а не технической реализации
2. Определите тип потребителя заранее, у каждого свои требования
3. Design-First: сначала контракт (OpenAPI), потом код. Это экономит время на изменения
4. Версионирование с самого старта. URI-версионирование (/v1/, /v2/) - простой и проверенный подход
5. Документация живет как часть продукта. OpenAPI + живые примеры + Changelog обязательны для публичных API
6. Закладывайте API Gateway. Контроль доступа + rate limiting + мониторинг.



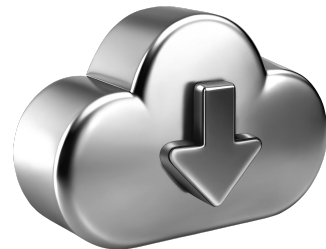
Теперь вы сможете:

Отправьте в чат номера достигнутых целей



Полезные материалы:

1. Статья про документацию API - <https://habr.com/ru/articles/722650/>
2. OpenAI API как стандарт отрасли - <https://openai.com/ru-RU/api/>
3. LiteLLM (AI Gateway) - <https://www.litellm.ai>



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

